LATIHAN TERSTRUKTUR (TUGAS KULIAH)

Pertemuan 4: Implementasi Unit Testing dengan GitHub Actions

Dari Konsep White-box ke Pipeline CI/CD Otomatis

Nama	Hafizh Zidan Handrian
NIM	1224002
Kelas / Institusi	Sistem Informasi — STMIK Bandung
Mata Kuliah	Pengujian Perangkat Lunak
Tanggal	13 Oktober 2026
Repository	github.com/Zidan-h4ndr1/latihan-git-1224002

Struktur dokumen: Cover • Bagian A (Git & GitHub) • Bagian B (Unit Testing) • Bagian C (GitHub Actions) • Bagian D (Coverage) • Bagian E (Bonus BankAccount) • Link Repository

Format file: Tugas04_GitHubActions_1224002_HafizhZidanHandrian.pdf • Diketik • Disertai project Maven + workflow di repository

BAGIAN A — GIT & GITHUB DASAR (25 POIN)

Soal 1 — Konsep Dasar (10 Poin)

1. Centralized VCS vs Distributed VCS

Aspek	Centralized (CVCS)	Distributed (DVCS)
Arsitektur	Satu server pusat menyimpan semua versi/history. Client hanya working copy.	Setiap developer punya salinan lengkap repo + history.
Ketergantungan server	Harus online untuk commit, log, diff history.	Bisa commit/branch/log offline; push/pull hanya saat sinkronisasi.
Single point of failure	Ya — server rusak berisiko hilang.	Tidak — setiap clone adalah backup penuh.
Branching/merging	Berat (server-side, mahal).	Ringan dan cepat (lokal dulu).
Contoh	1. Apache Subversion (SVN) 2. CVS / Perforce	1. Git 2. Mercurial

2. Lima Git commands wajib

• **git clone <url>** — menyalin remote repo ke lokal beserta seluruh history.

• **git add <file> / .** — memindahkan perubahan ke staging area.

• **git commit -m "pesan"** — menyimpan snapshot staged ke history lokal.

• **git push origin <branch>** — mengirim commit lokal ke remote (GitHub).

• **git pull origin <branch>** — mengambil + menggabungkan perubahan remote (fetch + merge). Pelengkap wajib: branch, checkout/switch, merge, log --oneline, status.

3. Fungsi .gitignore

File teks di root repo berisi pola file/direktori yang **diabaikan Git**. Mencegah file build (`target/`), IDE (`.idea/`, `.vscode/`), OS (`.DS_Store`), coverage (`*.exec`), dan secrets (`.env`) ter-commit. Repo menjadi bersih, kecil, dan aman. Template Maven ada di `.gitignore` repo ini.

Soal 2 — Praktik Git Commands (15 Poin)

Dieksekusi persis seperti history repo ini. Verifikasi dengan `git log --oneline --graph --all` dan `git branch -a`.

```
# 1. Buat repo di github.com/new  nama: latihan-git-1224002 (empty)
git clone https://github.com/Zidan-h4ndr1/latihan-git-1224002.git
cd latihan-git-1224002

git branch -M main
# buat README.md (nama + NIM 1224002)
git add README.md
git commit -m "docs: add README"
git push -u origin main

git branch feature-test
git checkout feature-test
# alternatif: git checkout -b feature-test

# buat test.txt
git add test.txt
git commit -m "test: add test file"
git push -u origin feature-test

git checkout main
git merge feature-test
git push origin main
```

Bukti: commit `docs: add README` hanya berisi README.md; commit `test: add test file` di branch feature-test hanya berisi test.txt; feature-test sudah di-merge ke main.

BAGIAN B — UNIT TESTING FRAMEWORK (35 POIN)

Soal 3 — Validasi Pendaftaran (15 Poin)

Kode: `src/main/java/com/stmik/PendaftaranValidator.java`

```
public String validasiPendaftaran(String nama, int umur, String email, boolean setujuSyarat) {
  if (nama == null || nama.isEmpty()) return "Nama harus diisi";            // P1
  if (umur < 17) return "Umur minimal 17 tahun";                             // P2
  if (email == null || !email.contains("@")) return "Email tidak valid";   // P3,P4
  if (!setujuSyarat) return "Harus menyetujui syarat dan ketentuan";         // P5
  return "Pendaftaran berhasil";
}
```

1. Lima test case (independent path)

TC	Path	Input	Expected
TC1	P1 — nama kosong	("", 20, "user@mail.com", true)	"Nama harus diisi"
TC2	P2 — umur < 17	("Budi", 16, "budi@mail.com", true)	"Umur minimal 17 tahun"
TC3	P3/P4 — email invalid	("Budi", 20, null, true) dan ("Budi", 20, "budimail.com", true)	"Email tidak valid"
TC4	P5 — tidak setuju	("Budi", 20, "budi@mail.com", false)	"Harus menyetujui syarat..."
TC5	Semua valid	("Budi", 20, "budi@mail.com", true)	"Pendaftaran berhasil"

P3/P4 dalam satu `if (||)` dipecah jadi 2 uji. File test berisi 7 @Test (termasuk varian null). Pola Arrange-Act-Assert.

2. Implementasi JUnit — `PendaftaranValidatorTest.java` — `mvn test -Dtest=PendaftaranValidatorTest`

```
@Test
public void testPath2_UmurKurangDari17() {
  String result = validator.validasiPendaftaran("Budi", 16, "budi@mail.com", true); // Act
  assertEquals("Umur minimal 17 tahun", result); // Assert
}
```

Soal 4 — StudentGrade (20 Poin)

1. Cyclomatic Complexity calculateGrade()

Predicate: `score<0, score>100, >=80, >=70, >=60, >=50` = 6. **V(G) = P + 1 = 7** (ketat McCabe, menghitung `||`). Jika `||` disederhanakan sebagai 1 keputusan if, maka **V(G) = 6** — nilai umum di kuliah (tiap if/else-if).

2. Independent Path

- Invalid → `throw IllegalArgumentException` (-1 / 101).
- `score >= 80` → "A" (85).
- `score >= 70` → "B" (75).
- `score >= 60` → "C" (65).
- `score >= 50` → "D" (55).
- Selain itu → "E" (30). Ketat: bedakan `<0` vs `>100`.

3. Unit test — `StudentGradeTest.java` (A/B/C/D/E, boundary 80/79, invalid; isPassed true/boundary 60/59/false/invalid)

```
@Test public void testCalculateGrade_A() {
  assertEquals("A", grade.calculateGrade(85));
}
@Test public void testCalculateGrade_InvalidHigh() {
  assertThrows(IllegalArgumentException.class, () -> grade.calculateGrade(101));
}
@Test public void testIsPassed_Boundary60() {
  assertTrue(grade.isPassed(60));
  assertFalse(grade.isPassed(59));
}
```

BAGIAN C — GITHUB ACTIONS (25 POIN)

Soal 5 — Analisis workflow Python CI (15 Poin)

1. Nama workflow: **Python CI** (`name: Python CI`).

2. Trigger: **push** ke **main, develop** dan **pull_request** ke **main**.

3. strategy.matrix: menjalankan **job yang sama paralel** untuk 3 versi Python (3.10, 3.11, 3.12) agar kompatibilitas terjamin tanpa duplikasi YAML; variabel `${{ matrix.python-version }}` dipakai di step setup.

4. Steps job test — soal menulis 5, YAML hanya memuat 4:

   - **Checkout code** (`actions/checkout@v4`).
   - **Setup Python ${{ matrix.python-version }}** (`actions/setup-python@v5`).
   - **Install dependencies** (`pip install pytest pytest-cov`).
   - **Run tests** (`pytest --cov=src --cov-report=xml`). Step ke-5 yang umum: Upload coverage report via `actions/upload-artifact`.

Soal 6 — Workflow Java (10 Poin) — `.github/workflows/java-test.yml`

```yaml
name: Java Unit Test

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Run tests
        run: mvn test

      - name: Generate coverage report
        run: mvn jacoco:report

      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco/
```

Memenuhi semua ketentuan: nama, trigger push main/develop + PR main, runner ubuntu-latest, 5 steps di atas.

BAGIAN D — CODE COVERAGE & INTEGRASI (15 POIN)

Soal 7 — Analisis coverage

```
src/calculator.py    10   0   100%
src/login_system.py  15   2    87%
src/payment.py       20   8    60%
src/utils.py          5   1    80%
TOTAL                50  11    78%
```

No	Pertanyaan	Jawaban
1	Total statement	**50** (kolom Stmts baris TOTAL).
2	Total miss	**11** statement tidak tereksekusi.
3	Coverage terendah	**src/payment.py — 60%**.
4	Menaikkan payment.py	`pytest --cov=src --cov-report=term-missing` untuk lihat baris miss; tambah test AAA untuk branch/exception/else yang belum tersentuh hingga >=80%.
5	Apakah 78% cukup?	**Belum.** Standar industri 80–90%. 11 statement belum teruji; payment.py (60%) berisiko tinggi (transaksi uang). Jadikan baseline, naikkan ke >=80% (ideal >=85%), prioritaskan payment.py.

BAGIAN E — STUDI KASUS TERPADU / BONUS (10 POIN)

Implementasi lengkap di repo ini:

• **Kode:** `src/main/java/com/stmik/BankAccount.java` — `deposit/withdraw/getBalance`; validasi amount > 0, withdraw tidak melebihi saldo, saldo awal tidak negatif (IllegalArgumentException).

• **Test:** `BankAccountTest.java` — 8 test (saldo awal 0, deposit valid, deposit negatif/nol ditolak, withdraw valid, withdraw berlebih ditolak, withdraw negatif ditolak, konstruktor negatif ditolak).

• **CI:** `.github/workflows/java-test.yml` — `mvn test`, `mvn jacoco:report`, upload `jacoco-report`.

• **Repo:** https://github.com/Zidan-h4ndr1/latihan-git-1224002 — tab Actions → Java Unit Test harus hijau; coverage: unduh artifact, buka `index.html` (atau `target/site/jacoco/` lokal).

```
mvn test
mvn jacoco:report
```

LINK REPOSITORY

https://github.com/Zidan-h4ndr1/latihan-git-1224002

Folder jawaban: `Tugas/Tugas04/` — file PDF ini + file .md. Pipeline: Actions → Java Unit Test. Coverage: artifact `jacoco-report`.

Dikerjakan oleh Hafizh Zidan Handrian (1224002) — 13 Oktober 2026.